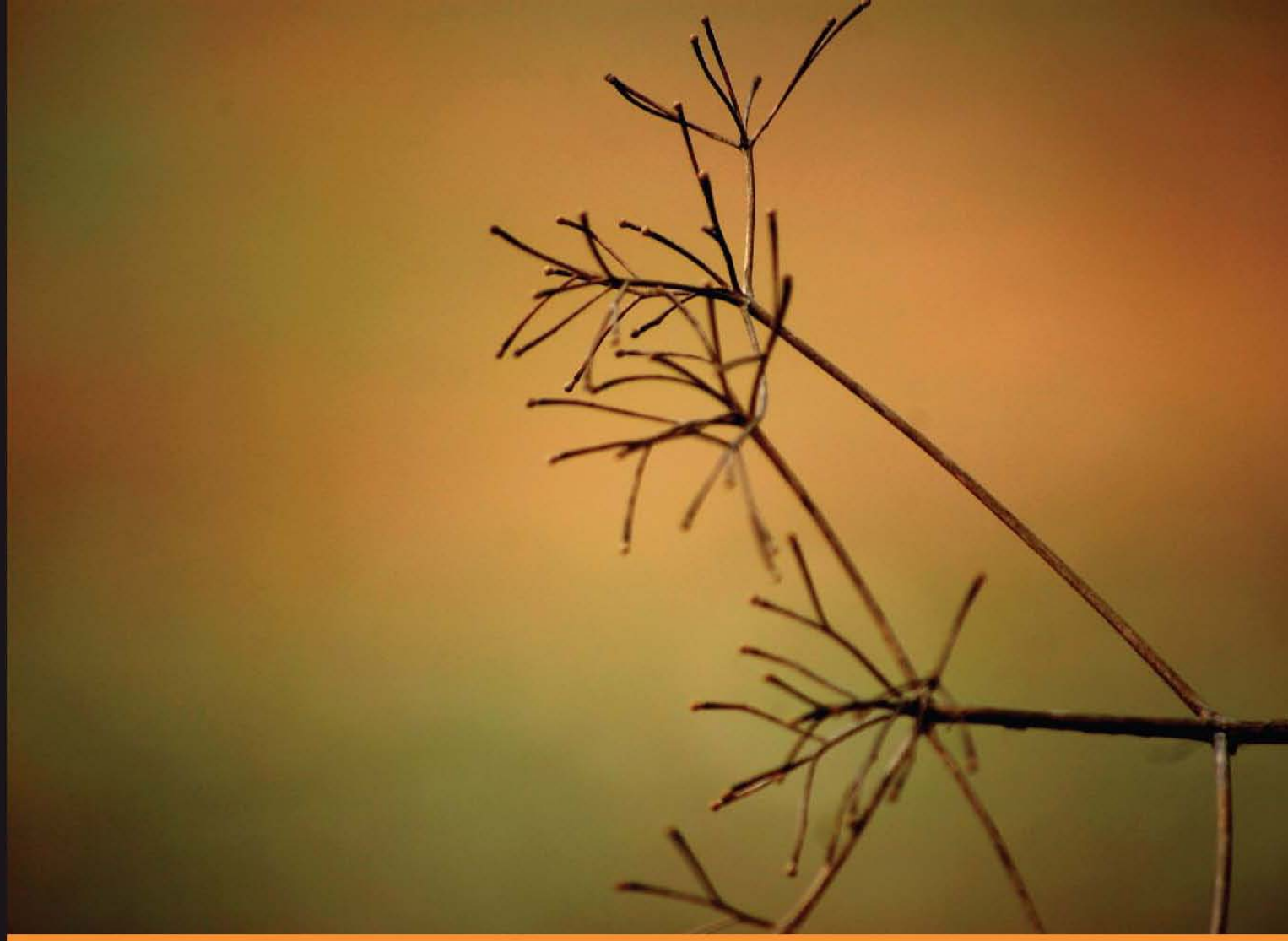

From Technologies to Solutions

Matplotlib for Python Developers

Build remarkable publication quality plots the easy way

Sandro Tosi

PACKT PUBLISHING

Matplotlib for Python Developers

Build remarkable publication quality plots the easy way

Sandro Tosi

BIRMINGHAM - MUMBAI

Matplotlib for Python Developers

First published: November 2009

Production Reference: 2221009

Published by Packt Publishing Ltd.
32 Lincoln Road
Olton
Birmingham, B27 6PA, UK.

ISBN 978-1-847197-90-0

www.packtpub.com

Cover Image by Raghuram Ashok (raghuram.ashok@gmail.com)

Credits

Author
Sandro Tosi

Reviewers
Michael Droettboom
Reinier Heeres

Acquisition Editor
Usha Iyer

Development Editor
Rakesh Shejwal

Technical Editor
Namita Sahni

Copy Editor
Leonard D'Silva

Indexers
Monica Ajmera
Hemangini Bari

Editorial Team Leader
Akshara Aware

Project Team Leader
Priya Mukherji

Project Coordinator
Zainab Bagasrawala

Proofreader
Lesley Harrison

Graphics
Nilesh Mohite

Production Coordinator
Adline Swetha Jesuthas

Cover Work
Adline Swetha Jesuthas

About the Author

Sandro Tosi was born in Firenze (Italy) in the early 80s, and graduated with a B.Sc. in Computer Science from the University of Firenze.

His personal passions for Linux, Python (and programming), and computer technology are luckily a part of his daily job, where he has gained a lot of experience in systems and applications management, database administration, as well as project management and development.

After having worked for five years as an EAI and an Application architect in an energy multinational, he's now working as a system administrator for an important European Internet company.

I'd like to thank Laura, who has assisted and supported me while writing this book.

About the Reviewers

Michael Droettboom holds a Master's Degree in Computer Music Research from The Johns Hopkins University. His research in optical music recognition lead to the development of the Gamera document image analysis framework, which has been used to recognize features in documents as diverse as medieval manuscript, Navajo texts, historical Scottish census data, and early American sheet music. His focus on computer graphics has lead to specializations in consumer electronics, computer-assisted engineering, and most recently, the science software for the Space Telescope Science Institute. He is currently one of the most active developers on the Matplotlib project.

I wish to thank my son, Kai, for asking all the hard questions.

Reinier Heeres has an MSc degree in Applied Physics from the Delft University of Technology, The Netherlands. He is currently pursuing a PhD there in the Quantum Transport group of the nanoscience department.

He has previously worked on Sugar, the child-friendly user interface mainly in use by One Laptop Per Child's $100 laptop. For this project, he designed the Calculator application.

Recently, he revived and extended the 3D plotting functionalities for Matplotlib to make it an excellent 2D graphing library, and a simple 3D plotting tool again.

Table of Contents

Preface

This book is about Matplotlib, a Python package for 2D plotting that generates production quality graphs. Its variety of output formats, several chart types, and capability to run either interactively (from Python or IPython consoles) and non-interactively (useful, for example, when included into web applications), makes Matplotlib suitable for use in many different situations.

Matplotlib is a big package with several dependencies and having them all installed and running properly is the first step that needs to be taken. We provide some ways to have a system ready to explore Matplotlib. Then we start describing the basic functions required for plotting lines, exploring any useful or advanced commands for our plots until we come to the core of Matplotlib: the object-oriented interface. This is the root for the next big section of the book—embedding Matplotlib into GUI libraries applications. We cannot limit it only to desktop programs, so we show several methods to include Matplotlib into web sites using low level techniques for two well known web frameworks—Pylons and Django. Last but not the least, we present a number of real world examples of Matplotlib applications.

The core concept of the book is to present how to embed Matplotlib into Python applications, developed using the main GUI libraries: GTK+, Qt 4, and wxWidgets. However, we are by no means limiting ourselves to that. The step-by-step introduction to Matplotlib functions, the advanced details, the example with web frameworks, and several real-life use cases make the book suitable for anyone willing to learn or already working with Matplotlib.

What this book covers

Chapter 1—Introduction to Matplotlib introduces what Matplotlib is, describing its output formats and the interactions with graphical environments. Several ways to install Matplotlib are presented, along with its dependencies needed to have a correctly configured environment to get along with the book.

Chapter 2—Getting started with Matplotlib covers the first examples of Matplotlib usage. While still being basic, the examples show important aspects of Matplotlib like how to plot lines, legends, axes labels, axes grids, and how to save the finished plot. It also shows how to configure Matplotlib using its configuration files or directly into the code, and how to work profitable with IPython.

Chapter 3—Decorate Graphs with Plot Styles and Types discusses the additional plotting capabilities of Matplotlib: lines and points styles and ticks customizations. Several types of plots are discussed and covered: histograms, bars, pie charts, scatter plots, and more, along with the polar representation. It is also explained how to include textual information inside the plot.

Chapter 4—Advanced Matplotlib examines some advanced (or not so common) topics like the object-oriented interface, how to include more subplots in a single plot or how to generate more figures, how to set one axis (or both) to logarithmic scale, and how to share one axis between two graphs in one plot. A consistent section is dedicated to plotting date information and all that comes with that. This chapter also shows the text properties that can be tuned in Matplotlib and how to use the LaTeX typesetting language. It also presents a section about contour plot and image plotting.

Chapter 5—Embedding Matplotlib in GTK+ guides us through the steps to embed Matplotlib inside a GTK+ program. Starting from embedding just the Figure and the Navigation toolbar, it will present how to use Glade to design a GUI and then embed Matplotlib into it. It also describes how to dynamically update a Matplotlib plot using the GTK+ capabilities.

Chapter 6—Embedding Matplotlib in Qt 4 explores how to include a Matplotlib figure into a Qt 4 GUI. It includes an example that uses Qt Designer to develop a GUI and how to use Matplotlib into it. What Qt 4 library provides for a real-time update of a Matplotlib plot is described here too.

Chapter 7—Embedding Matplotlib in wxWidgets shows what is needed to embed Matplotlib into a wxWidget graphical application. An important example is the one for a real-time plot update using a very efficient technique (borrowed from computer graphics), allowing for a high update rate. WxGlade is introduced, which guides us step-by-step through the process of wxWidgets GUI creation and where to include a Matplotlib plot.

Chapter 8—Matplotlib for the Web describes how to expose plots generated with Matplotlib on the Web. The first examples start from the lower ground, using CGI and the Apache mod_python module, technologies recommended only for limited or simple tasks. For a full web experience, two web frameworks are introduced, Pylons and Django, and a complete guide for the inclusion of Matplotlib with these frameworks is given.

Chapter 9—Matplotlib in the Real World takes Matplotlib and brings it into the real world examples field, guiding through several situations that might occur in the real life. The source code to plot the data extracted from a database, a web page, a parsed log file, and from a comma-separated file are described in full detail here. A couple of third-party tools using Matplotlib, NetworkX, and Mpmath, are described presenting some examples of their usage. A considerable section is dedicated to Basemap, a Matplotlib toolkit to draw geographical data.

What you need for this book

In order to be able to have the best experience with this book, you have to start with an already working Python environment, and then follow the advice in Chapter 1 on how to install Matplotlib and its most important dependencies. Some examples require additional tools, libraries, or modules to be installed: consult the distribution or project documentation for installation details.

Python, Matplotlib, and all other tools are cross-platform, so the book examples can be executed on Linux, Windows, or Mac OS X.

The book and the example code was developed using Python 2.5 and Matplotlib 0.98.5.3, but due to recent developments, Python 2.6 (Python 3.x is still not well supported by NumPy, Matplotlib, and several other modules) and Matplotlib 0.99.x can be used as well.

Who this book is for

This book is essentially for Python developers who have a good knowledge of Python; no knowledge of Matplotlib is required. You will be creating 2D plots using Matplotlib in no time at all.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "This is used for enhanced handling of the `datetime` Python objects."

A block of code is set as follows:

```
In [1]: import matplotlib.pyplot as plt
In [2]: import numpy as np
In [3]: y = np.arange(1, 3)
In [4]: plt.plot(y, 'y');
In [5]: plt.plot(y+1, 'm');
In [6]: plt.plot(y+2, 'c');
In [7]: plt.show()
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
c[0]*x**deg + c[1]*x**(deg - 1) + ... + c[deg]
```

Any command-line input or output is written as follows:

```
$ easy_install matplotlib-<version>-py<py version>-win32.egg
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "There are several aspects we might want to tune in a widget, and this can be done using the **Properties** window."

Warnings or important notes appear in a box like this.

Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an email to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a book that you need and would like to see us publish, please send us a note in the **SUGGEST A TITLE** form on `www.packtpub.com` or email `suggest@packtpub.com`.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book on, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code for the book

Visit `http://www.packtpub.com/files/code/7900_Code.zip` to directly download the example code.

The downloadable files contain instructions on how to use them.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration, and help us to improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/support`, selecting your book, clicking on the **let us know** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata added to any list of existing errata. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or web site name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1
Introduction to Matplotlib

A picture is worth a thousand words.

We all know that images are a powerful form of communication. We often use them to understand a situation better or to condense pieces of information into a graphical representation.

Just to give a couple of examples on how helpful they can be, let's consider the scientific and performance analysis fields. In order to clearly identify the bottlenecks, it is very important to be able to visualize data when analyzing performance information. Similarly, taking a quick glance at a graph drawn for a scientific experiment can give a scientist a better understanding of the results, something which is harder to achieve by looking only at the raw data.

Python is an interpreted language with a strong core functions basis and a powerful modular aspect which allows us to expand the language with external modules that offer new functionalities.

Modules reflect the Unix philosophy:

Do one thing, do it well.

So the result is that we have an extensible language with tools to accomplish a single task in the best possible way. Modules are often organized in *packages*. A package is a structured collection of modules that have the same purpose. One example of a package is **Matplotlib**.

Matplotlib is a Python package for 2D plotting that generates production-quality graphs. It supports interactive and non-interactive plotting, and can save images in several output formats (PNG, PS, and others). It can use multiple window toolkits (GTK+, wxWidgets, Qt, and so on) and it provides a wide variety of plot types (lines, bars, pie charts, histograms, and many more). In addition to this, it is highly customizable, flexible, and easy to use.

The dual nature of Matplotlib allows it to be used in both interactive and non-interactive scripts. It can be used in scripts without a graphical display, embedded in graphical applications, or on web pages. It can also be used interactively with the Python interpreter or **IPython**.

In this chapter, we will introduce Matplotlib, learn what it is, and what it can do. Later on, we will see what tools and Python modules are needed to have the best experience with Matplotlib and how to get them installed on our system, be it Linux, Windows, or Mac OS X.

The topics we are going to cover are:

- Introduction to Matplotlib
- Output formats and backends
- Dependencies
- How to install Matplotlib

Merits of Matplotlib

The idea behind Matplotlib can be summed up in the following motto as quoted by John Hunter, the creator and project leader of Matplotlib:

> *Matplotlib tries to make easy things easy and hard things possible.*

We can generate high quality, publication-ready graphs with minimal effort (sometimes we can achieve this with just one line of code or so), and for elaborate graphs, we have at hand a powerful library to support our needs.

Matplotlib was born in the scientific area of computing, where **gnuplot** and **MATLAB** were (and still are) used a lot.

With the entrance of Python into scientific toolboxes, an example of a workflow to process some data might be similar to this: "Write a Python script to parse data, then pass the data to a gnuplot script to plot it". Now with Matplotlib, we can write a single script to parse and plot data, with a lot more flexibility (that gnuplot doesn't have) and consistently using the same programming language.

We have to think of plotting not just as the final step in working with our data, but as an important way of getting visual feedback during the process. Here, the interactive capabilities of Matplotlib will come and rescue us.

Matplotlib was modeled on MATLAB, because graphing was something that MATLAB did very well. The high degree of compatibility between them made many people move from MATLAB to Matplotlib, as they felt like home while working with Matplotlib.

But what are the points that built the success of Matplotlib? Let's look at some of them:

- **It uses Python**: Python is a very interesting language for scientific purposes (it's interpreted, high-level, easy to learn, easily extensible, and has a powerful standard library) and is now used by major institutions such as NASA, JPL, Google, DreamWorks, Disney, and many more.
- **It's open source, so no license to pay**: This makes it very appealing for professors and students, who often have a low budget.
- **It's a real programming language**: The MATLAB language (while being Turing-complete) lacks many of the features of a general-purpose language like Python.
- **It's much more complete**: Python has a lot of external modules that will help us perform all the functions we need to. So it's the perfect tool to acquire data, elaborate the data, and then plot the data.
- **It's very customizable and extensible**: Matplotlib can fit every use case because it has a lot of graph types, features, and configuration options.
- **It's integrated with LaTeX markup**: This is really useful when writing scientific papers.
- **It's cross-platform and portable**: Matplotlib can run on Linux, Windows, Mac OS X, and Sun Solaris (and Python can run on almost every architecture available).

In short, Python became very common in the scientific field, and this success is reflected even on this book, where we'll find some mathematical formulas. But don't be concerned about that, we will use nothing more complex than high school level equations.

Matplotlib web sites and online documentation

The official Matplotlib presence on the Web is made up of two web sites:

- The SourceForge project page at `http://sourceforge.net/projects/matplotlib/`
- The main web site at `http://matplotlib.sourceforge.net/`

The SourceForge page contains, in particular, information about the development of Matplotlib, such as the released source code tarballs and binary packages, the SVN repository location, the bug tracking system, and so on. SourceForge also hosts some mailing lists for Matplotlib which are used for developers' discussions and users support.

On the main web site, we can find several important pieces of information about the Matplotlib package itself. For example:

- It contains a very attractive gallery with a huge number of examples of what Matplotlib can do
- The official documentation of Matplotlib is also present on this web site

The official documentation for Matplotlib is extensive. It covers in detail, all the submodules and the methods exposed by them, including all of their arguments. There are too many function arguments to cover in this book, so we are presenting only the most common ones here. In case of any doubts or questions, the official documentation is a good place to start your research or to look for an answer.

We encourage you to take a look at the gallery—it's inspiring!

Output formats and backends

The aim of Matplotlib is to generate graphs. So, we need a way to actually *view* these images or even to *save* them to files. We're going to look at the various output formats available in Matplotlib and the graphical user interfaces (GUIs) supported by the library.

Output formats

Given its scientific roots (that means several different needs), Matplotlib has a lot of output formats available, which can be used for articles/books and other print publications, for web pages, or for any other reason we can think of. Let's first differentiate the output formats into two distinct categories:

- **Raster images**: These are the classic images we can find on the Web or used for pictures. The most well known raster file formats are PNG, JPG, and BMP. They are widespread and well supported. The format of these images is like a matrix, with rows and columns, and at every matrix cell we have a pixel description (containing information such as colors). This format is said to be *resolution-dependent*, because the size of the matrix (the number of rows and columns) is determined when the image is created. An important parameter for raster images is the **DPI** (**dots-per-inch**) value. Once the image dimensions are decided (length and width, in inches), the DPI value specifies the detail level of the image. Hence, higher the DPI value, higher is the quality of the image (because for the same inch we get more dots). Scaling operations such as zooming or resizing can result in a loss of quality, because the image contains only a limited amount of information.
- **Vector images:** As opposed to raster images, vector images contain a *description* of the image in the form of mathematical equations and geometrical primitives (for example, points, lines, curves, polygons, or shapes). We can think of this format as a series of directives to plot the image: "Draw a point here, draw another point there, draw a line between those two points" and so on. Given this descriptive format, these images are said to be *resolution-independent*, because it's the image interpreter that replots the image at the requested resolution using the instructions in it. Typical examples of vector image usage are typesetting and CAD (architectural or mechanical parts drawings).

Of course, Matplotlib supports both the categories, particularly with the following output formats:

Format	Type	Description
EPS	Vector	Encapsulated PostScript.
JPG	Raster	Graphic format with lossy compression method for photographic output.
PDF	Vector	**Portable Document Format (PDF)**.
PNG	Raster	**Portable Network Graphics (PNG)**, a raster graphics format with a lossless compression method (more adaptable to line art than JPG).
PS	Vector	Language widely used in publishing and as printers jobs format.
SVG	Vector	**Scalable Vector Graphics (SVG)**, XML based.

PS or EPS formats are particularly useful for plots inclusion in LaTeX documents, the main scientific articles format since decades.

Backends

In the previous section, we saw the file output formats—they are also called **hardcopy backends** as they create something (a file on disk).

A backend that displays the image on screen is called a **user interface backend**.

The backend is that part of Matplotlib that works behind the scenes and allows the software to target several different output formats and GUI libraries (for screen visualization).

In order to be even more flexible, Matplotlib introduces the following two layers structured (only for GUI output):

- **The renderer***:* This actually does the drawing
- **The canvas***:* This is the destination of the figure

The standard renderer is the **Anti-Grain Geometry** (**AGG**) library, a high performance rendering engine which is able to create images of publication level quality, with anti-aliasing, and subpixel accuracy. AGG is responsible for the beautiful appearance of Matplotlib graphs.

The canvas is provided with the GUI libraries, and any of them can use the AGG rendering, along with the support for other rendering engines (for example, GTK+).

Let's have a look at the user interface toolkits and their available renderers:

Backend	Description
`GTKAgg`	GTK+ (The GIMP ToolKit GUI library) canvas with AGG rendering.
`GTK`	GTK+ canvas with GDK rendering. GDK rendering is rather primitive, and doesn't include anti-aliasing for the smoothing of lines.
`GTKCairo`	GTK+ canvas with Cairo rendering.
`WxAgg`	wxWidgets (cross-platform GUI and tools library for GTK+, Windows, and Mac OS X. It uses native widgets for each operating system, so applications will have the look and feel that users expect on that operating system) canvas with AGG rendering.
`WX`	wxWidgets canvas with native wxWidgets rendering.
`TkAgg`	Tk (graphical user interface for Tcl and many other dynamic languages) canvas with AGG rendering.

Backend	Description
`QtAgg`	Qt (cross-platform application framework for desktop and embedded development) canvas with AGG rendering (for Qt version 3 and earlier).
`Qt4Agg`	Qt4 canvas with AGG rendering.
`FLTKAgg`	FLTK (cross-platform C++ GUI toolkit for UNIX/Linux (X11), Microsoft Windows, and Mac OS X) canvas with Agg rendering.

Here is the list of renderers for file output:

Renderer	File type
AGG	`.png`
PS	`.eps` or `.ps`
PDF	`.pdf`
SVG	`.svg`
Cairo	`.png`, `.ps`, `.pdf`, `.svg`
GDK	`.png`, `.jpg`

The renderers mentioned in the previous table can be used directly in Matplotlib, when we want *only* to save the resulting graph into a file (without any visualization of it), in any of the formats supported.

We have to pay attention when choosing which backend to use. For example, if we don't have a graphical environment available, then we have to use the `AGG` backend (or any other file). If we have installed only the GTK+ Python bindings, then we can't use the `WX` backend.

About dependencies

As mentioned earlier, Matplotlib has its origin in scientific fields, so it is commonly used to plot huge datasets. Python's native support for long lists becomes impractical for such sizes, so Matplotlib needs better support for arrays.

NumPy, the de facto standard Python module for numerical elaborations, provides support for high performance operations even with big mathematical data types such as arrays or matrices—along with many other mathematical functions that can be useful to Matplotlib users.

NumPy has to be available to use Matplotlib.

Once we have chosen the set of **user interfaces (UIs)** we prefer, then we need to install the Python bindings for them. Here is a summarizing list:

User Interface (UI)	Binding	Version	Description
FLTK	pyFLTK	1.0 or higher	pyFLTK provides Python wrappers for the FLTK widgets library for use with `FLTKAgg` backend.
GTK+	PyGTK	2.2 or higher	PyGTK provides Python wrappers for the GTK+ widgets library to use it with the `GTK` or `GTKAgg` backend. It is recommended to use a version higher than 2.12, for a correct memory management.
Qt	PyQt or PyQt4	3.1 or higher and for Qt4, 4.0 or higher	PyQt or PyQt4 provides Python wrappers for the Qt toolkit and is required by the Matplotlib `QtAgg` and `Qt4Agg` backends. The library is widely used on Linux and Windows.
Tk	PyTK	8.3 or higher	Python wrapper for Tcl or Tk widgets library is used in `TkAgg` backend.
Wx	wxPython	2.6 or higher, or 2.8 or higher	wxPython provides Python wrappers for the wxWidgets library for use with the `WX` and `WXAgg` backends. It is widely used on Linux, Mac OS X, and Windows.

Another important tool, in particular for interactive usage, is IPython. It's an interactive Python shell with a lot of useful features, such as history, commands repeating, and others. It already has a **Matplotlib mode** in it. We'll be using IPython in this book, so it is recommended to install it.

Some of the tools that are needed by Matplotlib are already shipped with it (in the source code as well as in the binary distributions). Here is the list of those tools:

- **AGG (version 2.4)**: This is the Anti-Grain Geometry rendering engine. The local copy of the library is linked with the Matplotlib code in a static way. So, there's no need to install it (as a shared library).
- **pytz** (**version 2007g or higher**): This is used for handling the time zone for `datetime` Python objects. It will be installed if it's not already present in the system. It can be overridden using `setup.cfg`.
- **python-dateutil** (**version1.1 or higher**): This is used for enhanced handling of the `datetime` Python objects. It needs to be installed if it's not already present in the system and can be overridden using `setup.cfg`.

Build dependencies

The following tools are needed if we're going to install Matplotlib from the source:

- **Python**: Currently, only Python 2.x is supported (no Python 3 yet)
- **NumPy**: Version 1.1 or higher
- **libpng**: Version 1.1 or higher is needed to load or save PNG images (Windows users can skip this requirement)
- **FreeType**: Version 1.4 or higher is needed for reading **TrueType** font files (Windows users can skip this requirement)

libpng and FreeType for Windows users are already packaged in the Matplotlib Windows installer.

Installing Matplotlib

There are several ways to install Matplotlib on our system:

- Using packages from a Linux distribution
- Using binary installers (for Windows and Mac OS X only)
- Using packaged Python distributions that contain Matplotlib in the toolbox proposed
- From the source code

We will look at each option in detail. We assume that Python, NumPy, and the optional build and runtime dependencies are already installed in the system (in order to install them, refer to their installation guides).

Installing Matplotlib on Linux

The advantage of using a Linux distribution is that several programs and libraries are already prepared by the distribution developers and made available (in a package format) to users. All we have to do is use the right tool and install the package.

In the following table, we will present some of the common Linux distributions package names for Matplotlib and the tools we can use to install the package:

Distribution	Package name	Installer tool
Debian or Ubuntu	`python-matplotlib`	Synaptic (graphical)
(and all other Debian derivatives)		apt-get or aptitude (command line)
Fedora	`python-matplotlib`	PackageKit (graphical)
		yum or rpm (command line)
openSUSE	`python-matplotlib`	YaST (graphical)
		zipper or rpm (command line)

Installing Matplotlib on Windows

Before we can install Matplotlib, we have to satisfy its main dependencies. So, we have to download:

- Installers for Python, which are available in the **DOWNLOAD** section of `http://www.python.org/`
- Installers for NumPy, which are available in the **Download** section of `http://www.scipy.org/`

Once we've got the above packages correctly installed, we can go to the main project page of Matplotlib on SourceForge at `http://sourceforge.net/projects/matplotlib/`. In the **Files** section, we can find the relative versions of the binary packages for the Python that we have just installed (2.4, 2.5, or 2.6).

Installing Matplotlib on Mac OS X

The procedure to install Matplotlib correctly on Mac OS X is similar to that of Windows.

First of all, we need to download:

- Installers for Python, which are available in the **DOWNLOAD** section of `http://www.python.org/` (the one already available in Mac OS X fives problem when using Matplotlib)
- Installers for NumPy, which are available in the **Download** section of `http://www.scipy.org/` or can be retrieved directly from `http://pythonmac.org/`

At this point, once they are correctly installed, we can download the binary installer from the download area of Matplotlib SourceForce page at `http://sourceforge.net/projects/matplotlib/` or we can retrieve the version available at `http://pythonmac.org/`.

Installing Matplotlib using packaged Python distributions

There are some packaged distributions of Python that contain Matplotlib in them, along with many other tools, such as IPython, NumPy, SciPy, and so on. These distributions will set up all the necessary things we need so that we can use Matplotlib on our machine. Some of the distributions are as follows:

- **Enthought Python Distribution** (**EPD**): This package is available for Windows, Mac OS X, and Red Hat. We can download it from `http://www.enthought.com/products/epd.php`.
- **Python(x,y)**: This package is available for Windows and Ubuntu at `http://www.pythonxy.com/`.
- **Sage**: This package is available for Linux at `http://www.sagemath.org/`.

These are mainly scientific distributions that install a lot of tools we don't directly need or use, but they have the advantage of making it easy to get Python, NumPy, and Matplotlib installed and working on our system.

Installing Matplotlib from source code

There are two ways of obtaining the Matplotlib source code. They are:

- Downloading it from the source code tarballs available in the download area of Matplotlib SourceForge project page at `http://sourceforge.net/projects/matplotlib/`.
- Retrieving it from the Subversion (SVN) repository. This is the place where development takes place, so use it only if you know what you're doing.

If we decided to go with SVN, we can follow the instructions available in the **Develop** section of `http://sourceforge.net/projects/matplotlib/`.

If we are going to use the source code tarball, we will have to unpack it, go into the created source directory, and execute the following commands:

```
$ python setup.py build
$ sudo python setup.py install
```

These commands will build and then install Matplotlib. We will need administrative privileges to install it into the system directories (hence the `sudo` command in this Linux example).

Many aspects of the installation can be tuned using `setup.cfg`, a file shipped with the source code and used at build and install time. We can use it to customize the build process, such as changing the default backend, or choosing whether to install the optional libraries or not.

If we want to install Matplotlib from source on Windows, the **Files** section of Matplotlib SourceForge page contains handy `egg` files which we can download (choosing the Python version of interest) and then install using `setuptools` command. The following command will install Matplotlib on your machine:

```
$ easy_install matplotlib-<version>-py<py version>-win32.egg
```

`Egg` files are also available for Mac OS X, and we can use them in the same way as described above.

Testing our installation

To ensure we have correctly installed Matplotlib and its dependencies, a very simple test can be carried out in the following manner:

```
$ python
Python 2.5.4 (r254:67916, Feb 18 2009, 03:00:47)
[GCC 4.3.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> print numpy.__version__
1.2.1
>>> import matplotlib
>>> print matplotlib.__version__
0.98.5.3
```

If there's no error while executing this, then we are done.

Summary

In this chapter, we have covered the following areas:

- What is Matplotlib and what are its main key points
- The several file output formats and graphical user interfaces (GUIs) that are supported
- The packages required by Matplotlib, and the ones needed for the GUI bindings
- Installing and testing Matplotlib on a Linux, Windows, or Mac OS X system, in multiple ways

At this point, we only have a general idea of what Matplotlib is, along with the package correctly installed in our system. So let's go and start using Matplotlib!